

LaunchLabs: An Agentic AI System for Athletic Performance Analysis

Project Proposal

1. Problem Statement & Motivation

What real-world business problem are you addressing?

Strength and conditioning coaches managing multiple athletes face a significant challenge: analyzing complex jump performance data, identifying training weaknesses, and generating personalized workout programs. Currently, this process requires manual data interpretation, cross-referencing sport-specific benchmarks, and custom program design—consuming 2-4 hours per athlete weekly. Coaches often lack the expertise to identify bilateral asymmetries that indicate injury risk, or to properly sequence exercises for optimal adaptation.

Who are the target users?

The primary users are athletic performance coaches working with 10-50 athletes across sports requiring explosive power: basketball, volleyball, and track & field. Secondary users include physical therapists monitoring injury rehabilitation and sports scientists conducting research on training adaptations. Other users I plan to work with are independent adult athletes that do not have access to OVR Jumps or their own coaches.

Why is an agentic AI solution appropriate here?

An agentic AI solution is ideal because the task requires:

1. multi-step reasoning across interconnected data points (jump height, ground contact time, bilateral asymmetry, sport-specific benchmarks)
2. dynamic decision-making based on individual athlete profiles rather than fixed workflows

3. iterative refinement of training programs based on progress data
4. integration across multiple data sources (OVR Jump, video analysis, training logs). A static workflow cannot adapt to the variability in athlete needs, training phases, and injury histories.

2. Use Cases & Scenarios

Scenario 1: New Athlete Assessment

A coach uploads OVR Jump data for a new volleyball player. The Agentic system:

- (a) parses CSV data to extract jump metrics
- (b) retrieves sport/position-specific benchmarks
- (c) calculates bilateral asymmetry from single-leg tests
- (d) identifies that the athlete has 18% force asymmetry (high injury risk)
- (e) generates an initial assessment report with visualizations
- (f) recommends unilateral corrective exercises before beginning plyometric training.

Scenario 2: Training Program Generation

A coach requests a 6-week program for an intermediate basketball guard. The system:

- (a) retrieves the athlete's historical data and current metrics
- (b) analyzes weaknesses (low RSI, poor approach bonus)
- (c) queries the exercise library for appropriate progressions
- (d) generates a periodized program with strength, power, and plyometric blocks
- (e) includes warm-up/cool-down protocols
- (f) creates a downloadable PDF with exercise descriptions and coaching cues.

Scenario 3: Progress Monitoring & Adaptation

After 4 weeks, the coach uploads new test results. The system:

- (a) compares current vs. baseline metrics
- (b) calculates week-over-week improvement rates
- (c) identifies that RSI improved but bilateral asymmetry worsened
- (d) alerts the coach to potential overtraining on the dominant leg
- (e) automatically adjusts the program to increase unilateral work
- (f) generates a progress report with trend visualizations and recommendations

3. Initial System Design

Agent Architecture:

- **Orchestrator Agent:** Coordinates workflow, maintains conversation state, routes to specialized agents
- **Data Analyst Agent:** Parses CSV/Excel uploads, calculates metrics (RSI, GCT, asymmetry), validates data quality
- **Benchmark Agent:** Retrieves sport/position-specific standards, compares athlete metrics, classifies performance levels
- **Program Generator Agent:** Selects exercises from library, applies periodization logic, generates structured workout plans
- **Visualization Agent:** Creates charts (radar, line, bar), generates PDF reports, formats data for dashboard display
- **Critic Agent:** Validates recommendations against safety guidelines, flags injury risks, ensures exercise progressions are appropriate

Tools & Data Sources:

- File parsing: Python pandas for CSV/Excel, custom parsers for OVR Jump exports

- Storage: Notion API for athlete profiles and training logs
- Visualization: PowerBI for interactive dashboards, ReportLab for PDF generation
- Knowledge base: Curated exercise library (200+ exercises with metadata), sport benchmarks database
- External: Web search for latest research on training protocols (rate-limited)

4. Feasibility & Risks

Potential Challenges:

- **Hallucination:** Risk of generating unsafe exercise recommendations. The critic agent validates all outputs against safety rules; exercises must exist in a curated library.
- **Inconsistent settings:** Volleyball players I plan on testing play on different settings: indoor, grass, beach and their setting of choice will have different force impact due to how much force the ground will absorb, taking away from the jump height. My solution is to keep the setting as consistent as possible and to have a data column stating where the data tracking is and what ground it is on, so that the setting is a known factor.
- **Inconsistent clients:** Clients may be inconsistent in their training that is given to them that the agent will give based off of their first initial testing. So some clients will have missing data, while other more consistent clients will build more data. As long as they are honest with how their training is going in between metric testing, this should be a minor problem.

Constraints:

- **Rate limits:** Notion API (3 requests/second); batch operations for bulk updates
- **Privacy:** Athlete data must remain confidential; no external sharing without consent

- **Computation:** Complex visualizations may require client-side rendering to avoid timeout

Timeline:

- Week 1-2: Core agent architecture, data parsing pipeline, benchmark database, testing metric testing myself, and look for potential athletes interested to track data
- Week 3-4: Program generation logic, exercise library integration, basic visualizations, build testing client list and follow up for more testing
- Week 5-6: Critic agent, safety validation, progress monitoring features
- Week 7-8: Integration testing, user feedback incorporation, final documentation